

Do Subscribe and Support ❤️ *****

Cloud Engineer Interview Questions *****

Important Note:

You will find AWS services naming convention here. If you are preparing for Azure or GCP, you can still use these questions—just change the names to match your cloud provider.

All the very best – Keep Hustling and don't lose hope

LEVEL 3 – Scenario-Based Questions (Most Important)

(Thinking + Troubleshooting + Architecture)

1. Production website is down. What are your first 5 steps?

First, I would confirm the outage by checking from multiple locations and verifying monitoring alerts to understand whether the issue is global or region-specific. Next, I would check the load balancer health to see if it is receiving traffic and whether backend targets are healthy or failing health checks. Then, I would verify the status of compute resources such as EC2 instances or Kubernetes pods to ensure they are running and not repeatedly restarting. After that, I would review any recent changes such as deployments, configuration updates, scaling events, or security changes that could have caused the outage. Finally, I would analyze logs and metrics to identify errors or unusual patterns while keeping stakeholders informed about progress and impact.

2. EC2 is running but the website is not accessible. How do you debug?

If the EC2 instance is running but the website is unreachable, I would start by checking security groups to confirm inbound traffic on ports 80 or 443 is allowed from the correct sources. Then, I would validate network routing by reviewing the subnet's route table, ensuring it has a path to the internet through an Internet Gateway or Load Balancer. Next, I would log into the instance and verify that the web server process is running and listening on the expected port. I would check application and web server logs for errors and confirm that the instance is passing health checks if it is behind a load balancer. Finally, I would inspect NACLs, DNS records, and SSL configuration to rule out networking or certificate issues.

3. CPU utilization suddenly spikes to 95%. What will you do?

I would first determine whether the CPU spike is temporary or sustained by reviewing historical metrics and recent trends. Then, I would identify which process or application component is consuming CPU using instance or container-level monitoring. If the spike is due to increased traffic, I would allow Auto Scaling or horizontal pod scaling to add capacity. If it is caused by a bug, memory leak, or inefficient code path, I would mitigate by restarting the service and investigating logs and recent deployments. I would also evaluate whether scaling thresholds, instance sizes, or resource limits need adjustment to prevent similar incidents in the future.

4. Traffic increases 5x unexpectedly. What happens in your architecture?

In a resilient architecture, the load balancer would distribute the increased traffic across healthy instances without manual intervention. Auto Scaling groups or Kubernetes would detect the increased load and provision additional instances or pods to handle demand. Since the application is stateless, scaling can occur without session loss, while caches such as Redis or CloudFront reduce load on backend services. Databases would handle the increased read load through read replicas or scaling, and monitoring systems would alert the team so they can observe behavior and take corrective action if needed.

5. How would you design a highly available web application?

I would design the application to run across multiple Availability Zones to avoid single points of failure. Traffic would be routed through a managed load balancer that performs health checks and directs requests only to healthy instances. The application layer would be stateless and use Auto Scaling to adapt to changing load. Data would be stored in managed databases with Multi-AZ replication, and static assets would be served through S3 and a CDN. Continuous monitoring, automated backups, and tested failover procedures would ensure high availability and fast recovery.

6. How would you migrate an on-prem application to cloud?

I would begin by analyzing the application's architecture, dependencies, data volume, and performance requirements. Based on this assessment, I would choose an appropriate migration strategy such as lift-and-shift for speed or re-platforming for optimization. I would provision cloud infrastructure in advance, migrate data using secure transfer methods, and validate functionality in a staging environment. Once testing is complete, I would plan a phased or low-downtime cutover with rollback options. After migration, I would optimize for scalability, security, and cost.

7. Database is slow. How do you identify the bottleneck?

I would start by checking database performance metrics such as CPU usage, memory, disk I/O, and connection counts to identify resource constraints. Then, I would review slow query logs to detect inefficient or long-running queries. I would analyze whether the slowdown is caused by increased traffic, missing indexes, locking, or application behavior such as excessive queries. I would also evaluate network latency and caching effectiveness. Based on findings, I would optimize queries, add indexes, introduce caching, or scale the database as needed.

8. Users complain about slow performance. What will you check?

I would first correlate user reports with monitoring data to determine when and where latency increases are occurring. Then, I would check frontend performance, backend response times, and database metrics to isolate the layer causing the slowdown. I would examine logs for errors or timeouts and verify third-party integrations are functioning correctly. I would also review recent deployments or configuration changes that could have introduced regressions. This layered approach helps identify the root cause efficiently.

9. Kubernetes pod is in CrashLoopBackOff. Steps to debug?

I would start by describing the pod to review events and identify why it is restarting. Then, I would check container logs to understand whether the application is crashing due to configuration errors, missing dependencies, or runtime failures. I would validate environment variables, secrets, and mounted volumes. I would also check resource limits

to see if the pod is being terminated due to memory or CPU constraints. If needed, I would reproduce the issue in a test environment before applying a fix.

10. How do you implement zero downtime deployment?

Zero downtime deployment is achieved by ensuring that new versions of the application are introduced gradually while the old version continues serving traffic. This can be done using rolling deployments, blue-green deployments, or canary releases. Load balancers and health checks ensure traffic is only routed to healthy instances. Database changes are handled carefully to maintain backward compatibility. If issues arise, the deployment can be paused or rolled back quickly, ensuring users experience no disruption.

11. How would you reduce cloud cost in a running project?

I would start by understanding where the money is being spent by reviewing cost reports and identifying the top services contributing to the bill. Then I would look for underutilized resources such as oversized EC2 instances, unused EBS volumes, idle load balancers, and unattached Elastic IPs. I would right-size compute by downsizing instances or moving to auto scaling so capacity matches demand. I would also review storage usage, move infrequently accessed data to cheaper tiers, and delete unused snapshots. Finally, I would evaluate long-term workloads for Reserved Instances or Savings Plans and ensure non-production environments are shut down when not in use.

12. Private EC2 cannot connect to RDS. How will you troubleshoot?

I would first confirm that the EC2 instance and RDS database are in the same VPC or properly connected VPCs. Then I would check the RDS security group to ensure it allows inbound traffic on the database port from the EC2 instance's security group. Next, I would verify that the EC2 instance's security group allows outbound traffic to the database port. I would also confirm the database endpoint, port, and credentials are correct. If networking looks fine, I would check NACL rules, route tables, and DNS resolution to ensure traffic can flow correctly within the VPC.

13. How do you design disaster recovery for production?

I design disaster recovery based on business requirements like RTO and RPO. First, I identify critical components such as databases, application servers, and external dependencies. I ensure data is backed up regularly and replicated to another region where required. Depending on the criticality, I choose a DR strategy such as backup-and-restore, pilot light, warm standby, or active-active. Infrastructure is defined using IaC so it can be recreated quickly, and regular DR drills are performed to validate recovery procedures and timelines.

14. A new deployment caused errors in production. What will you do?

First, I would assess the impact by checking monitoring dashboards, error rates, and user reports. If the issue is severe, I would immediately stop the rollout or roll back to the previous stable version to restore service. Then I would analyze logs, metrics, and recent code changes to identify the root cause. I would verify whether the issue is related to configuration, database changes, or compatibility problems. Once fixed, I would redeploy carefully and add safeguards such as canary deployments or better testing to prevent recurrence.

15. How would you secure a fintech/payment application architecture?

I would design the architecture using defense-in-depth. Network isolation would be enforced using private subnets, strict security groups, and limited ingress points. IAM would follow least privilege with strong authentication and MFA. Sensitive data would be encrypted both at rest and in transit using managed key services. Secrets would be stored securely and rotated regularly. I would enable extensive logging, monitoring, and alerting, and ensure compliance with industry standards like PCI-DSS. Regular security testing and audits would also be part of the design.

Topic: Networking & Connectivity Issues

16. A private EC2 cannot access the internet. How will you debug?

I would first check whether the private subnet's route table has a default route pointing to a NAT Gateway or NAT instance. Then I would confirm the NAT Gateway is in a public

subnet with a route to an Internet Gateway. Next, I would check security groups to ensure outbound traffic is allowed and verify NACL rules for outbound and inbound traffic. I would also confirm DNS resolution is working. If all networking looks correct, I would test connectivity from the instance to isolate whether the issue is routing, security, or application-level.

17. Users cannot access your public website. What layers will you check?

I would check this layer by layer, starting from DNS to the application. First, I would verify DNS records are correct and resolving properly. Then I would check whether the load balancer is reachable and has healthy targets. Next, I would verify security groups and NACLs allow inbound traffic. I would then check whether the application is running and responding correctly on backend instances. Finally, I would inspect logs and monitoring data to identify errors or failures at any layer.

18. Traffic is reaching Load Balancer but not EC2. What could be wrong?

If traffic reaches the load balancer but not EC2, I would first check whether the EC2 instances are healthy in the target group. Then I would verify the instance security group allows inbound traffic from the load balancer's security group on the correct port. I would also confirm that the application is listening on the expected port and path used for health checks. Misconfigured health checks, incorrect ports, or security group rules are common causes of this issue.

19. EC2 can ping internally but cannot connect to database. Why?

If EC2 can ping other instances but cannot connect to the database, it usually indicates a port or permission issue rather than basic connectivity. I would check whether the database security group allows inbound traffic from the EC2 security group on the database port. I would also verify outbound rules on the EC2 security group. If those are correct, I would check NACLs and confirm the database endpoint and port are correct. Authentication issues or database-level connection limits could also be contributing factors.

20. You connected two VPCs but instances cannot communicate. What will you verify?

I would first verify that the VPC peering or Transit Gateway connection is active. Then I would check route tables in both VPCs to ensure each has routes to the other VPC's CIDR block pointing to the peering connection. Next, I would review security groups and NACLs to ensure traffic is allowed. I would also confirm there is no CIDR overlap between the VPCs, as overlapping CIDRs prevent routing.

21. Route tables look correct but traffic still not flowing. What next?

If route tables are correct, I would move to the next layers. I would check security groups to ensure traffic is allowed on the required ports and protocols. Then I would review NACL rules, paying attention to both inbound and outbound rules since NACLs are stateless. I would verify DNS resolution and test connectivity using tools like ping or curl from within the instance. Finally, I would look at flow logs and monitoring data to identify where traffic is being dropped.

Topic: Scaling & Performance

22. Application slows down during peak hours. How will you investigate?

I would start by correlating the slowdown with traffic patterns to confirm whether it aligns with peak usage. Then I would analyze metrics across all layers, including load balancer latency, application response time, CPU, memory, and database performance. I would check whether Auto Scaling or pod scaling is triggering as expected and whether new capacity is actually handling traffic. Next, I would review database metrics and slow queries to see if the backend is becoming a bottleneck. Finally, I would examine caching efficiency, connection limits, and any third-party dependencies that might degrade under load.

23. Auto Scaling is configured but instances are not scaling. Why?

I would first verify whether scaling policies are correctly configured and attached to the Auto Scaling Group. Then I would check whether the metrics used for scaling, such as CPU or request count, are actually crossing the defined thresholds. I would also review cooldown periods and scaling limits to ensure scaling is not being blocked. Next, I would

confirm that the Auto Scaling Group has permission to launch instances and that there are no capacity issues such as instance type shortages or subnet IP exhaustion. Lastly, I would review activity logs to identify any failed scaling attempts.

24. CPU is low but response time is high. What might be the issue?

Low CPU with high response time usually indicates a bottleneck outside of compute. I would check database latency, slow queries, and connection pool exhaustion. Then I would look at network latency, load balancer queues, and external service calls. I would also examine thread pools, locks, and application-level resource limits that may cause waiting even when CPU is idle. Caching issues or inefficient I/O operations are also common causes. This situation often requires application-level profiling rather than infrastructure scaling.

25. One Availability Zone goes down. What should happen in a properly designed system?

In a properly designed system, traffic should automatically shift away from the failed Availability Zone without manual intervention. The load balancer should stop routing traffic to unhealthy targets in that AZ, and Auto Scaling should launch replacement instances in healthy AZs. The application should remain available because it is deployed across multiple AZs. Databases configured with Multi-AZ or replication should fail over automatically. Users may experience brief degradation, but the service should continue running.

26. Users from another country complain about latency. How do you reduce it?

I would first identify where latency is occurring by analyzing request paths and geographic traffic distribution. To reduce latency, I would introduce a CDN to cache static and dynamic content closer to users. I would also consider deploying the application in multiple regions and using geo-based routing to direct users to the nearest region. Optimizing API responses, enabling compression, and reducing payload sizes would further improve performance. For global databases, read replicas closer to users may also help.

Topic: Architecture & Design

27. Design a highly available 3-tier architecture.

I would design the system with a presentation layer, application layer, and database layer, each distributed across multiple Availability Zones. Traffic would enter through a managed load balancer in front of the web tier. The application tier would be stateless and run in an Auto Scaling Group across AZs. The database tier would use a managed database with Multi-AZ replication or clustering. Shared storage and static content would be served from managed services like object storage and CDN. Monitoring, backups, and automated recovery would be built into every layer.

28. How would you eliminate single points of failure in a setup?

I would start by identifying components that exist in only one instance, such as a single server, database, or NAT gateway. Each critical component would be duplicated across Availability Zones or replaced with a managed, highly available service. Load balancers would be used to distribute traffic, and Auto Scaling would ensure failed instances are replaced automatically. Databases would use replication and automated failover. I would also ensure DNS, networking, and authentication systems are resilient and monitored.

29. How would you design a system that can handle sudden traffic spikes?

I would design the system to be stateless so it can scale horizontally. Auto Scaling or container orchestration would be configured to respond quickly to traffic spikes using appropriate metrics. A load balancer would distribute traffic evenly, while caching layers would absorb repeated requests. Rate limiting and throttling would protect backend systems. The database would be scaled using read replicas or partitioning, and monitoring would be in place to alert the team during abnormal spikes.

30. How would you design backup and restore strategy for critical data?

I would classify data based on criticality and define RPO and RTO requirements. Backups would be automated, encrypted, and stored in multiple locations or regions. For databases, I would enable point-in-time recovery and periodic full snapshots. Backups would be tested regularly by restoring them to non-production environments. I would also document restore procedures and ensure access to backups is restricted and audited.

31. How would you ensure minimal downtime during maintenance?

I would perform maintenance using rolling updates or blue-green deployments so traffic is never fully interrupted. Load balancers would route traffic only to healthy instances, and new instances would be validated before receiving traffic. For database maintenance, I would rely on managed services that support maintenance windows and automatic failover. Clear communication and monitoring would ensure issues are detected quickly, and rollback plans would always be ready.

32. How would you design secure access for internal employees?

I would design access using centralized identity management with strong authentication and least-privilege permissions. Employees would authenticate using SSO and MFA, and access would be granted through roles rather than individual credentials. Sensitive systems would be placed in private networks accessible only through secure gateways such as VPN or identity-aware proxies. Access would be logged and audited, and permissions would be reviewed regularly to prevent privilege creep.

Topic: Security & IAM

33. A developer accidentally made an S3 bucket public. What will you do?

First, I would immediately restrict access by blocking public access at the bucket level to stop any further exposure. Then, I would review bucket policies and ACLs to identify exactly what was changed and revert it to a secure configuration. Next, I would assess impact by checking access logs to determine whether sensitive data was accessed or downloaded. I would rotate any exposed credentials or invalidate cached data if necessary. Finally, I would implement preventive controls such as SCPs, automated policy checks, and alerts to prevent public exposure in the future.

34. How would you implement least privilege access across teams?

I would start by understanding team responsibilities and mapping required permissions for each role. Access would be granted using IAM roles rather than individual users, with policies scoped to specific services, actions, and resources. Broad permissions and wildcards would be avoided, and permission boundaries would be used where teams need flexibility. Temporary access would be granted for elevated privileges instead of permanent permissions. Regular audits and access reviews would be conducted to remove unused permissions and prevent privilege creep.

35. An EC2 was compromised. What immediate steps will you take?

First, I would isolate the instance by removing it from load balancers and restricting network access to prevent lateral movement. I would preserve evidence by taking snapshots of the instance and attached volumes for forensic analysis. Then, I would rotate credentials, revoke compromised keys, and review IAM roles attached to the instance. I would analyze logs to understand how the compromise occurred and identify affected systems. Finally, I would rebuild the instance from a known secure image rather than attempting to clean it.

36. How would you securely allow application to access database credentials?

I would store database credentials in a managed secrets service rather than hard-coding them in the application. The application would retrieve credentials dynamically at runtime using an IAM role with least-privilege access. Credentials would be encrypted at rest and in transit, and automated rotation would be enabled where supported. The application would be designed to handle credential rotation without downtime. Access to secrets would be logged and monitored to detect misuse.

37. How would you audit who accessed sensitive resources?

I would enable centralized logging and auditing across all accounts and services. Access events would be captured using audit logs that record who accessed which resource, when, and from where. I would store logs in a secure, immutable location and use

monitoring tools to query and analyze access patterns. Alerts would be configured for suspicious or unauthorized access attempts. Regular reviews and reports would be generated to meet compliance and security requirements.

Topic: Storage & Database

38. Database storage is full. What are your options?

First, I would assess urgency by checking whether the database is read-only or completely unavailable. Short-term options include increasing storage size, freeing up space by deleting unused data, or archiving old records. Long-term solutions involve optimizing data retention policies, cleaning up logs, and improving indexing or partitioning. I would also evaluate whether the database needs to be scaled vertically or horizontally. Alerts would be configured to prevent the issue from recurring.

39. How would you migrate data from one region to another safely?

I would start by assessing data size, consistency requirements, and acceptable downtime. Data would be transferred using secure, encrypted methods while validating integrity before cutover. For databases, replication or snapshot-based migration would be used to keep data in sync until the final switch. I would test the migrated data in the target region before redirecting traffic. Once confirmed, I would perform a controlled cutover and monitor closely for issues.

40. How would you handle backup failure alerts?

I would first verify whether the alert represents a real failure or a transient issue. Then, I would identify which backup failed and why, such as permission issues, storage limits, or service disruptions. I would immediately rerun the backup or initiate a manual backup if necessary. Next, I would fix the root cause and test backup and restore functionality. Finally, I would review alerting thresholds and redundancy to ensure future failures are detected early.

Topic: Monitoring & Troubleshooting

41. Application is throwing random 500 errors. How do you approach it?

I would begin by correlating error occurrences with logs, metrics, and traffic patterns to identify trends. Then, I would check application logs to determine whether errors originate from the application, database, or external dependencies. I would review recent deployments or configuration changes that could have introduced instability. I would also examine resource usage, connection pools, and timeout settings. Based on findings, I would fix the root cause and add better monitoring to catch similar issues earlier.

42. Logs show high memory usage but CPU is fine. What does that indicate?

High memory usage with low CPU typically indicates memory leaks, excessive caching, or inefficient memory management rather than compute bottlenecks. I would analyze memory allocation patterns and garbage collection behavior. I would also check whether the application is holding onto objects longer than necessary or loading large datasets into memory. Container or instance memory limits might need adjustment, but the root cause should be fixed at the application level.

43. Users report intermittent failures but metrics look normal. What next?

When metrics look normal but users experience issues, I would look deeper into logs, traces, and edge cases that metrics may not capture. I would analyze request-level logs to identify patterns such as timeouts, retries, or regional issues. I would check dependencies such as third-party APIs or DNS resolution. Distributed tracing would help track failures across services. This approach often reveals issues hidden beneath aggregated metrics.

44. How would you design centralized logging for cloud resources?

I would design centralized logging by collecting logs from applications, infrastructure, and managed services into a single logging platform. Logs would be structured and tagged with metadata such as service name, environment, and request ID. Retention and access controls would be enforced to balance cost and compliance. Search and alerting

capabilities would be enabled to support troubleshooting and security investigations. This ensures logs are easily accessible and actionable.

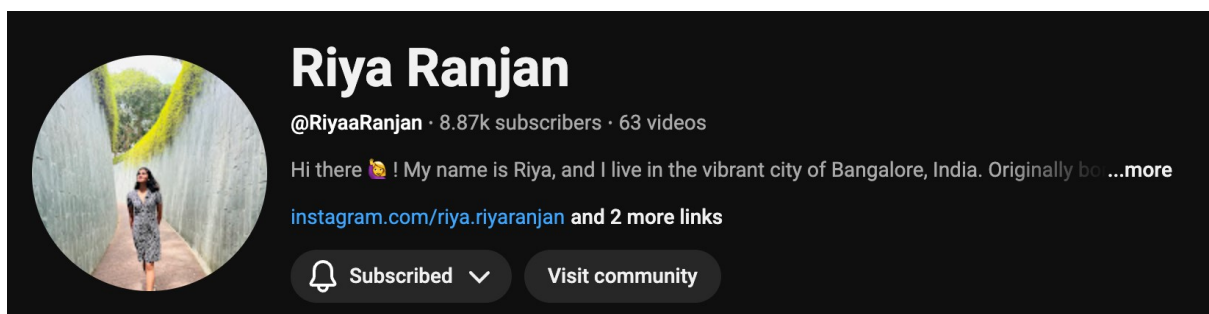
45. How do you ensure alerts are meaningful and not noisy?

I would design alerts based on symptoms that impact users rather than raw metrics. Alerts would be tied to thresholds that indicate real problems, such as sustained error rates or latency increases. I would use aggregation, suppression, and severity levels to reduce noise. Alerts would be tested regularly to ensure they trigger correctly and include enough context for quick action. This ensures teams trust alerts and respond effectively.

Before checking the answers, Test yourself and try to answer these questions

*******(without cheating 🤞)*******

Don't forget to Subscribe and Support



It means a lot! 💕